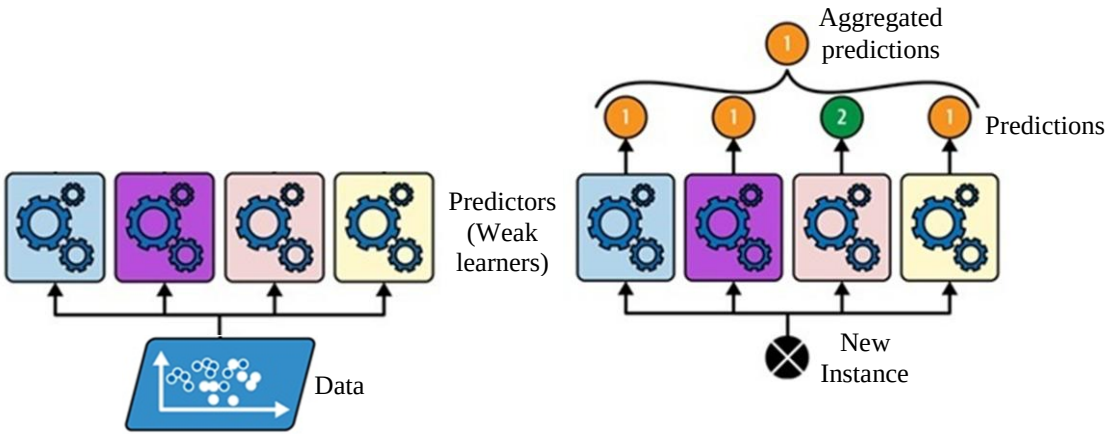
Ensemble Learning

Chapter 7: **Ensemble Learning and Random Forest**
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Ensemble Learning

- Ensemble learning combines predictions from multiple machine learning models to produce more accurate and robust results than a single model.
- This idea is similar to the wisdom of the crowd, where combining opinions from many individuals often gives better results than relying on one expert.
- In an ensemble:
  - Individual models are called base learners or weak learners.
  - The combined model is called a strong learner because it usually achieves better accuracy and generalization performance.
- After training, predictions from all models are aggregated:
  - Classification typically uses majority voting.
  - Regression typically uses averaging.
- Ensemble methods often improve performance by reducing overfitting, variance, or bias.
- Even if individual learners perform only slightly better than random guessing, combining many sufficiently diverse learners can create a strong learner.
- Greater diversity reduces correlation between predictors and lowers ensemble variance.
- Diversity among learners can be achieved by:
  - Using different machine learning algorithms
  - Training on different subsets of data
  - Using different feature subsets
- Popular ensemble methods include voting classifiers, bagging and pasting, random forests, boosting, and stacking ensembles.



Voting Classifiers

- Voting classifiers combine predictions from multiple diverse learners to produce a more accurate and robust model.
- In voting classifiers, multiple models are usually trained on the same dataset, while diversity is mainly achieved using different learning algorithms.
- In a hard voting classifier, each classifier votes for a class label.
  The class receiving the majority of votes becomes the ensemble's final prediction.
- In a soft voting classifier, the base classifiers estimate class probabilities.
  The predicted probabilities are averaged, and the class with the highest average probability is selected.
  Soft voting generally performs better because it considers prediction confidence.

Bagging and Pasting

- Bagging and pasting train the same learning algorithm on different random subsets of the training data to produce diverse predictors.
- Two ways of sampling data are bagging and pasting.
- If sampling is performed with replacement, the method is called bagging (bootstrap aggregating).
  The same training instance can appear multiple times in the subset for a predictor
- If sampling is performed without replacement, the method is called pasting.
  Each training instance can appear only once in the subset.
- Each predictor is trained independently, so training and prediction can be performed in parallel using multiple CPU cores or servers. This makes bagging highly scalable and efficient.

Out-of-Bag (OOB) Evaluation

- Since each model is trained on a bootstrap sample, some training instances are left out during training.
- These unused samples are called Out-of-Bag (OOB) samples.
- OOB samples can be used as a validation set to estimate model performance without requiring separate validation data.

Ensembles Based on Decision Trees

- Although decision trees are simple and interpretable, they have several limitations.
  - They use a greedy approach while selecting splits, which may not always produce the globally optimal tree.
  - They can easily overfit the training data, especially when the tree becomes very deep.
  - They have high variance and are highly sensitive to small changes in the training data; a slight variation in the dataset may produce a completely different tree structure.
- However decision trees work very well in ensembles.
- Decision tree–based ensembles are among the most effective machine learning methods for a wide range of classification and regression tasks.
- Random Forests and Gradient Boosting are examples of decision tree based ensembles.

Random Forests

- A random forest is an ensemble learning method consisting of many decision trees trained on random subsets (bagging or pasting) of data and features.
- Instead of relying on one unstable tree, many trees are trained on different bootstrap samples, and their predictions are combined; the resulting ensemble usually generalizes much better than a single decision tree.
- The scheme works as follows:
  1. Multiple random subsets of the training data are created using bagging or pasting.
  2. A separate decision tree is trained on each bootstrap sample.
  3. At each split, only a random subset of features is considered instead of all features.
  4. Each tree makes its own prediction.
  5. The final prediction is obtained using majority voting for classification, and averaging for regression.
- If all trees always choose the same strongest feature first, the trees become very similar; random feature selection forces different trees to learn different patterns from the data. This increases diversity and improves ensemble performance.

Boosting

- Boosting converts weak learners into a strong learner by training models sequentially, where each new model focuses on correcting the mistakes of the previous ones. Final predictions are made using a weighted combination of all models.
- Unlike bagging and random forests, boosting models are not trained independently.
- Instead, each model depends on the previous one, which means:
  - Training cannot be parallelized.
  - It is generally slower and less scalable than bagging.
- Despite this, boosting often achieves very high predictive performance.
- There are many boosting methods available, but the most popular are AdaBoost and Gradient Boosting.

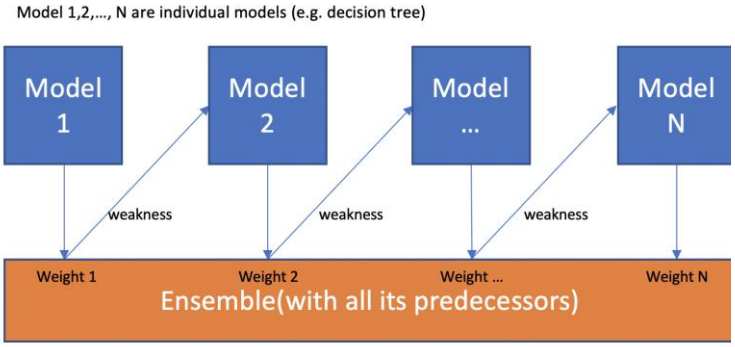
AdaBoost (Adaptive Boosting)

- AdaBoost is one of the most widely used boosting algorithms.
- AdaBoost builds a strong model by sequentially training weak learners, where each new learner focuses more on the mistakes of previous ones through updated sample weights, and the final prediction is obtained using a weighted combination of all learners.
- AdaBoost often uses simple base learners such as shallow decision trees (decision stumps).

Algorithm

- Train a base learner on the original dataset, where each sample is assigned an equal weight initially.
- Evaluate predictions on the training set to:
  - assign an overall weight to the learner.
  - increase the weights of misclassified samples.
- Train the next learner using the updated sample weights, forcing it to focus more on hard-to-classify samples.
- Repeat the process for several learners.
- The final prediction is obtained by combining all models using a weighted voting scheme, where more accurate learners have greater influence and weaker learners have less influence.

- Incorporation of weighted samples directly affects the learning process of a learner.
  For example, decision trees now use weighted impurity measures (weighted gini impurity or weighted entropy), so that splits are chosen to better classify high-weight samples.



Gradient Boosting

- Gradient Boosting is a boosting technique in which learners are trained sequentially, where each new learner attempts to reduce the errors (residuals) made by the previous learners.
- Each new learner is trained on the residual errors produced by the previous ensemble of learners. The final prediction is obtained by adding the outputs of all learners together.
- Early learners correct large errors, while later learners make smaller refinements.
- Typically uses shallow decision trees as weak learners.
- XGBoost (Extreme Gradient Boosting) is an optimized and highly efficient implementation of Gradient Boosting. It improves speed, scalability, and predictive performance through several enhancements such as regularization, parallel processing, and optimized tree construction.
- A learning rate α is applied to each learner's output to control the overall learning speed of the model.

Algorithm

- Train an initial learner on the original dataset.
- Calculate the residual errors between actual and predicted values.
- Train the next learner using these residual errors as target values.
- Add the new learner's predictions to the previous predictions to improve the overall model.
- Repeat the process until the residual errors become very small.
- Hence the final prediction is $\hat{y} = \widehat{y1} + \alpha\widehat{y2} + \alpha\widehat{y3} + \cdots = \widehat{y1} + \alpha\sum_{t=2}^{T}\widehat{yt}$, where T is the number of learners (trees).

Example: Training Process with Decision Trees

- Suppose we want to predict house prices using Gradient Boosting.

Sample	Actual Price
1	100
2	150
3	200

- We train 3 trees: Tree 1, Tree 2, and Tree 3.
  Tree 1 is trained on original target values.
  Tree 2 is trained on residuals from Tree 1.
  Tree 3 is trained on residuals from Tree 2.

Tree 1:

Sample	**Actual Output** $y1 = y$	**Tree 1 output** $\widehat{y1}$	**Predicted Price** $\hat{y} = \widehat{y1}$	**Residual Error** $r1 = y - \hat{y}$
1	100	90	90	100 – 90 = 10
2	150	130	130	150 – 130 = 20
3	200	170	170	200 – 170 = 30

Tree 2:

Sample	**Residual Error** $y2 = r1$	**Tree 2 output** $\widehat{y2}$	**Updated Prediction** $\hat{y} = \widehat{y1} + \widehat{y2}$	**Residual Error** $r2 = y - \hat{y}$
1	10	8	90 + 8 = 98	100 – 98 = 2
2	20	15	130 + 15 = 145	150 – 145 = 5
3	30	25	170 + 25 = 195	200 – 195 = 5

Tree 3:

Sample	**Residual Error** $y3 = r2$	**Tree 3 output** $\widehat{y3}$	**Updated Prediction** $\hat{y} = \widehat{y1} + \widehat{y2} + \widehat{y3}$	**Residual Error** $r3 = y - \hat{y}$
1	2	1	98 + 1 = 99	100 – 99 = 1
2	5	4	145 + 4 = 149	150 – 149 = 1
3	5	4	195 + 4 = 199	200 – 199 = 1

- With learning rate final prediction looks like: $\hat{y} = \widehat{y1} + \alpha\widehat{y2} + \alpha\widehat{y3}$